

tuf-go-tuf

Helix OTA — Stack Research: TUF + go-tuf/v2 (supply-chain integrity for OTA artifacts)

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Evaluation of The Update Framework (TUF) and the <code>github.com/theupdateframework/go-tuf/v2</code> library as the metadata/signing layer that wraps Helix OTA's already-signed update artifacts. Covers the four top-level roles, the attack classes TUF mitigates, the maturity of the go-tuf/v2 metadata + client API, how TUF layers over the native A/B <code>payload.bin/.zip</code> artifacts, and the adoption effort for the Go control plane. Library facts verified live against the GitHub API and <code>pkg.go.dev</code> on 2026-06-07. go-tuf/v2 <code>metadata/updater</code> and <code>metadata/trustedmetadata</code> client-flow APIs could not be fully enumerated from <code>pkg.go.dev</code> in this pass (the package index rendered the constructor/repository surface but truncated the client subpackages). Adoption-effort figures are estimates, not measured. Android-side TUF client maturity is the weakest link and is flagged below.
Issues	
Issues summary	Server-side (Go) story is strong and verifiable; device-side (Android) TUF client story needs a dedicated follow-up

Field	Value
Fixed	because go-tuf/v2 is a Go library, not an Android one. initial research note
Fixed summary	Roles, attack classes, library version/license/maturity, and metadata API surface verified against primary sources. Unverifiable specifics are marked “UNVERIFIED — needs confirmation”.
Continuation	Feed into ADR on supply-chain security; pair with the artifact-signing research note and the Android client integration spec. Spike a minimal go-tuf/v2 repo + refresh client before committing the dependency. Open a separate investigation for the on-device TUF client (Go-on-Android via the engine, or a hand-rolled minimal verifier).

Table of contents

- [§1. Scope & question](#)
- [§2. What TUF is](#)
- [§3. The four top-level roles](#)
- [§4. Attack classes mitigated](#)
- [§5. go-tuf/v2 — facts, maturity, API surface](#)
- [§6. How TUF layers over signed OTA artifacts](#)
- [§7. Effort to adopt in Helix OTA](#)
- [§8. Risks, gaps & open questions](#)
- [§9. Recommendation](#)
- [§10. Sources](#)

§1. Scope & question

Helix OTA already plans per-artifact integrity: SHA-256 + a detached signature, verified server-side on upload and device-side before `update_engine.applyPayload`. The open question is whether to add **TUF** on top, and if so whether go-tuf/v2 is the right implementation for the Go control plane. TUF does not replace artifact signing — it answers a different question: “*can a client trust the metadata that tells it which artifact is current, and can it keep trusting it after a key or a mirror is compromised?*” This note evaluates roles, mitigated attacks, go-tuf/v2 maturity, the layering model, and adoption cost.

§2. What TUF is

TUF (The Update Framework) is a CNCF-graduated specification for securing software-update systems. Its core idea is **separation of trust across multiple signing roles with independent keys, expiration, and version pinning**, so that compromise of any one key (especially online keys) does not let an attacker push arbitrary, stale, or inconsistent updates. The current spec version pinned by go-tuf/v2 is `SPECIFICATION_VERSION = "1.0.31"` (from the package constants). TUF is metadata-centric: clients fetch a small set of signed JSON metadata files, validate them in a fixed order, and only then trust the hashes/lengths that point at the actual artifacts.

§3. The four top-level roles

Quotes below are from the TUF specification (theupdateframework.github.io/specification/latest). All four are signed roles; `root` is the trust anchor and is the only role shipped/pinned out-of-band.

Role	Responsibility (spec)	Key posture
root	“delegates trust to specific keys trusted for all other top-level roles.” The trust anchor; distributes/rotates the public keys of all roles.	Offline, highest security; threshold-signed.
targets	“signs metadata that describes” the target files (the artifacts) — names, lengths, hashes — and “can delegate full or partial trust to other roles.”	Offline preferred; can delegate to per-product/per-channel roles.
snapshot	“signs a metadata file that provides information about the latest version of all targets metadata on the repository.” Prevents inconsistent (mix-and-match) metadata views.	Can be online (e.g. CI-signed).
timestamp	“periodically signs a timestamped statement containing the hash of the snapshot file.” Proves freshness and bounds replay windows.	Online; “the risk posed to clients by the compromise of this key is minimal.”

Client verification order is fixed: `root` → `timestamp` → `snapshot` → `targets` (then delegated targets). **Delegations** (incl. hash-bin and succinct hash-bin / TAP-15) let targets hand off subtrees of the target namespace to other keys — directly useful for Helix’s per-device-group / per-channel artifact partitioning.

§4. Attack classes mitigated

From the TUF spec's security goals (short quotes). These are the concrete classes that justify TUF *on top of* plain artifact signing:

- **Arbitrary software installation** — attacker cannot “provide arbitrary files in response to download requests.”
- **Rollback attacks** — cannot “trick clients into installing software that is older than that which the client previously knew to be available.” (Version pinning across metadata.)
- **Fast-forward attacks** — cannot “arbitrarily increase the version numbers of metadata files” (which would otherwise poison rollback protection after recovery).
- **Indefinite freeze attacks** — cannot serve “the same, outdated metadata without the client being aware” (timestamp expiry bounds this).
- **Endless data attacks** — cannot respond with “huge amounts of data” (lengths are pinned in metadata).
- **Slow retrieval attacks** — spec addresses trickle-feeding data to stall clients. (UNVERIFIED — phrasing not re-quoted in this pass; spec lists it under the same security section.)
- **Extraneous dependencies attacks** — cannot cause “clients to download or install software dependencies that are not the intended dependencies.”
- **Mix-and-match attacks** — cannot “trick clients into using a combination of metadata that never existed together” (snapshot role).
- **Wrong software installation** — cannot “provide a file that is not the one the client wanted.”
- **Malicious mirrors preventing updates** — “a repository mirror cannot prevent updates from good mirrors.”
- **Key compromise resilience** — compromising “a single key or less than a given threshold of keys” cannot “compromise clients” (threshold signing + offline root/targets).

Plain SHA-256 + per-artifact signature covers *wrong/arbitrary file* only if the signing key is intact and the device already knows the correct hash. It does **not** cover rollback, freeze, mix-and-match, mirror-denial, or graceful recovery from key compromise. That delta is the case for TUF.

§5. go-tuf/v2 — facts, maturity, API surface

Verified facts (GitHub API + pkg.go.dev, 2026-06-07):

- Module path: `github.com/theupdateframework/go-tuf/v2` (v2 is a major-version module; import path includes /v2).
- License: **Apache-2.0** (verified via GitHub API `license.spdx_id`).
- Latest release: **v2.4.2**, published **2026-05-19** (verified via `gh api .../releases`). Preceding: v2.4.1 (2026-01-26), v2.4.0 (2026-01-21), v2.3.1 (2026-01-19), v2.3.0 (2025-11-05).
- Stars: **705**; not archived; last push **2026-06-03** (active). (Star count is a weak proxy and recorded only for context.)
- Pinned spec version: **1.0.31**.
- Crypto: ED25519, RSA (e.g. `rsassa-pss-sha256`), ECDSA.

- Features: consistent snapshots; standard + hash-bin delegations; **succinct hash-bin delegations (TAP-15)**; **TAP-4** multi-repository consensus; unrecognized-field preservation (forward-compat).

Maturity: go-tuf/v2 is a ground-up redesign of the legacy go-tuf v0.7.0 (which the project describes as hard to maintain), modeled on python-tuf's architecture. The project states it is "used in production by various tech companies and open-source organizations." That production claim is the project's own (not independently audited here) — treat as **UNVERIFIED** — **needs confirmation** for any specific adopter list. The API is generics-based and clean.

Metadata API surface (verified on pkg.go.dev):

- Constructors: `Root(...)` `*Metadata[RootType]`, `Targets(...)`, `Snapshot(...)`, `Timestamp(...)`.
- Signing/verify: `(*Metadata[T]).Sign(signer signature.Signer)`, `VerifyDelegate(role, meta)`, `ClearSignatures()`.
- (De)serialization: `ToBytes/FromBytes`, `ToFile/FromFile`.
- Targets helpers: `TargetFiles.FromFile/FromBytes` (computes length+hashes), `VerifyLengthHashes(data)`.
- Root key admin: `RootType.AddKey/RevokeKey`, `IsExpired`.
- Delegations: Delegations with Roles `[]DelegatedRole`, `SuccinctRoles`, `GetRolesForTarget(path)`, `DelegatedRole.IsDelegatedPath(path)`.
- Keys: `KeyFromPublicKey`, `Key.ID()`, `Key.ToPublicKey()`. Signer integration is via `secure-systems-lab/go-securesystemslib signature.Signer`.
- Subpackages: `metadata`, `metadata/config`, `metadata/fetcher`, `metadata/trustedmetadata`, `metadata/updater`, `metadata/multirepo`.

The **client refresh + download flow** lives in `metadata/updater` (ngclient-style `Updater.Refresh()`, target lookup, `DownloadTarget`) built on `metadata/trustedmetadata` (the trusted set enforcing the root→timestamp→snapshot→targets order). The exact signatures of these two subpackages were **not** fully captured in this pass (pkg.go.dev truncated them) — **UNVERIFIED** — **needs confirmation**; verify against `metadata/updater` examples before estimating client code.

§6. How TUF layers over signed OTA artifacts

TUF is **additive and non-invasive** to the existing artifact pipeline. The artifacts themselves (the `payload.bin` inside the A/B OTA `.zip`, or the full `.zip`) remain byte-identical and keep their own AOSP/payload signature; TUF treats each as an opaque **target file** identified by path + length + SHA-256/SHA-512 hash.

Proposed layering for Helix OTA:

1. **Build/publish (Go control plane):** when a release is uploaded to MinIO/S3, the server computes `TargetFiles.FromFile(...)`, adds the entry to a targets (or delegated per-channel) metadata, bumps snapshot and timestamp, signs each with the appropriate signer, and publishes the metadata next to the blobs.
2. **Trust roots:** root and targets keys held offline (HSM/KMS, ceremony-rotated); snapshot/timestamp can be online (signed by the control plane or CI). timestamp re-signed on the OTA poll cadence to bound freeze windows.

3. **Distribution:** metadata served over the existing HTTP/3→HTTP/2 + Brotli path; consistent-snapshot naming makes it CDN/cache-friendly.
4. **Device:** client fetches metadata, runs TUF verification (role order, expiry, version monotonicity, threshold sigs), resolves the target hash, then downloads the artifact and verifies length+hash **before** handing payload.bin to `update_engine.applyPayload`. The native A/B engine's own payload signature check remains as defense-in-depth.

Net: TUF wraps the metadata/trust decision; the A/B engine still does the atomic, anti-bricking apply + automatic boot-failure rollback. They are complementary, not overlapping.

§7. Effort to adopt in Helix OTA

All figures are **estimates, not measured** — flagged accordingly.

Server-side (Go) — LOW-to-MEDIUM. go-tuf/v2 is idiomatic Go, Apache-2.0, single dependency tree (relies on `go-securesystemslib`). The publish path (build targets/snapshot/timestamp, sign, write) is a few hundred lines wrapping the verified constructor + Sign + delegation APIs. Hooks cleanly into the existing upload/MinIO flow. Estimated 1–2 engineer-weeks for a first integration + tests. **UNVERIFIED — needs confirmation** via a spike.

Key management — MEDIUM-to-HIGH (process, not code). The real cost is operational: offline root/targets key custody, a documented signing ceremony, threshold + rotation policy, and timestamp re-signing automation. This is inherent to TUF, not to go-tuf, and is where most adopters spend effort.

Device-side (Android) — HIGH / OPEN. This is the critical gap. go-tuf/v2 is a **Go** library; the Android client is Kotlin + a foreground service. Options, all unproven for Helix here: - (a) Run go-tuf/v2 on-device via a Go-built native lib (gomobile/JNI) — adds a Go toolchain + binary-size cost to the APK. - (b) Hand-roll a minimal TUF client in Kotlin that implements only the refresh/verify flow against our metadata — full control, but security-sensitive code we'd own. - (c) Use an existing JVM/Android TUF client — **none verified to exist or be maintained; UNVERIFIED — needs confirmation**. This decision dominates the adoption cost and must be its own ADR/spike before committing.

§8. Risks, gaps & open questions

- **Device client is the long pole.** No verified production-grade Android/Kotlin TUF client found. Resolve before adopting TUF as a hard requirement.
- **Operational burden of offline keys.** TUF's guarantees depend on root/targets keys being offline and threshold-signed; weak key custody negates most benefits. Needs an HSM/KMS + ceremony plan.
- **metadata/updater API not fully verified** in this pass — confirm the Updater/TrustedMetadata signatures against current examples before sizing client work.
- **Production-adopter and “minimal risk on timestamp compromise” claims** are the project's/spec's own wording — fine to cite, not independently audited here.

- **Spec slow-retrieval phrasing** not re-quoted — minor; confirm if cited verbatim downstream.
- **Star count (705)** is context only; do not use as a maturity gate.

§9. Recommendation

Adopt TUF as the metadata/trust layer for the server-side publish pipeline now; gate device-side enforcement behind a dedicated Android-client spike. go-tuf/v2 (v2.4.2, Apache-2.0, actively maintained, spec 1.0.31, generics API, TAP-4/TAP-15 support) is a sound, low-risk choice for the Go control plane and materially closes attack classes that plain per-artifact signing leaves open (rollback, freeze, mix-and-match, mirror-denial, key-compromise recovery). The blocker is not the library but the absence of a verified Android TUF client and the operational key-management process. Recommended sequence: (1) prototype the server publish + a Go reference refresh client to validate the model end-to-end; (2) ADR + spike for the on-device client (gomobile vs. hand-rolled Kotlin); (3) define the key-custody ceremony; (4) only then make device-side TUF verification mandatory. Keep the existing SHA-256 + artifact signature and the A/B engine's payload check as defense-in-depth regardless.

Overall confidence: medium. Library facts (version, date, license, API, maturity signals) are HIGH (verified against GitHub API + pkg.go.dev). Roles/attacks are HIGH (TUF spec). Adoption-effort numbers and the Android-client path are LOW-to-MEDIUM (estimates + an unresolved gap).

§10. Sources

Consulted 2026-06-07:

- TUF specification (roles, attack classes): <https://theupdateframework.github.io/specification/latest/>
- TUF project homepage: <https://theupdateframework.io/>
- go-tuf repository (README, status, features): <https://github.com/theupdateframework/go-tuf>
- go-tuf releases (version + dates, verified via GitHub API repos/theupdateframework/go-tuf/releases): <https://github.com/theupdateframework/go-tuf/releases>
- go-tuf repo metadata (stars=705, license=Apache-2.0, pushed 2026-06-03, archived=false), verified via GitHub API repos/theupdateframework/go-tuf.
- go-tuf/v2 metadata package API (constructors, Sign/VerifyDelegate, TargetFiles, Delegations, subpackages, SPEC 1.0.31): <https://pkg.go.dev/github.com/theupdateframework/go-tuf/v2/metadata>
- TAP-4 (multi-repo consensus) and TAP-15 (succinct hash-bin delegations): TUF TAPs — <https://github.com/theupdateframework/taps> (referenced via go-tuf feature list; TAP text not re-fetched this pass — UNVERIFIED for exact TAP wording).